

FICHA DE AVALIAÇÃO

Laboratório de Programação Java · Desafio: Cadastro de Dicionário de Ambientes

Aluno(a):		Turma:	
Data:		Avaliador:	Prof. Rômulo

1. O desafio em uma frase

Criar um programa que funcione como uma **agenda de ambientes**: você informa uma **chave** (ex.: **F07**) e o programa guarda a **descrição** (ex.: "Laboratório de Programação Java"), salvando tudo em um arquivo **.txt**.

Chave	Descrição
F07	Laboratório de Programação Java
B03	Sala de Aula Padrão
G09	Oficina de Lanternagem e Pintura

O programa deve ter um **menu dentro de um do while** com as opções: **cadastrar, listar, pesquisar, excluir, alterar e sair**. Use Map (interface) e HashMap (classe).

2. Antes de começar

Materiais de apoio permitidos: aula de Lógica Aristotélica; FAQ *faq_logica.pdf* do repositório; VisuAlgo (visualgo.net/en); CSVistool (csvistool.com); GeeksforGeeks (geeksforgeeks.org); seu fluxograma ou pseudocódigo feito em aula.

Dica para quem está começando: antes de digitar qualquer código, desenhe o fluxograma do menu. Se você consegue explicar o desenho, o código fica muito mais fácil.

3. Níveis de desempenho (valem para cada capacidade)

Nível	Descrição	Pontos
4 · Pleno	Faz sozinho(a), funciona e explica com clareza	3,10
3 · Satisfatório	Funciona, com pequenas ajudas ou explicação incompleta	2,30
2 · Em desenvolvimento	Faz parcialmente, precisa de ajuda frequente	1,50
1 · Inicial	Tentou, mas não funciona ou não consegue explicar	0,70
0 · Não demonstrou	Não apresentou	0,00

Cada capacidade tem duas partes: **Elaborar** (fazer no código) e **Explicar** (dizer com suas próprias palavras, oralmente). Quem só copia e não explica não atinge a nota máxima.

4. Capacidades avaliadas (3,10 pontos cada)

Capacidade 1 · try / catch / finally (Seg)

- ☐ Coloca o código que pode dar erro dentro do try (ex.: abrir arquivo, converter texto em número)
- ☐ Trata o erro no catch com uma mensagem amigável
- ☐ Usa o finally para o que **sempre** deve acontecer (ex.: fechar o arquivo)

Pergunta oral: O que acontece com o programa se o arquivo não existir e você não usar o try/catch?

Analogia: O try é tentar atravessar a rua; o catch é o plano B se vier um carro; o finally é olhar para os dois lados sempre.

Nível: ☐ 0 ☐ 1 ☐ 2 ☐ 3 ☐ 4

Pontos: _____

Capacidade 2 · Swing (JOptionPane, JFrame ou outros)

- ☐ Usa `JOptionPane.showInputDialog` para receber dados
- ☐ Usa `JOptionPane.showMessageDialog` para exibir resultados
- ☐ Monta o menu de opções em janela (ou JFrame) e o usa dentro do loop

Pergunta oral: Qual a diferença entre `showInputDialog` e `showMessageDialog`?

Nível: ☐ 0 ☐ 1 ☐ 2 ☐ 3 ☐ 4

Pontos: _____

Capacidade 3 · DateTimeFormatter (Ter)

- ☐ Cria um formato com `DateTimeFormatter.ofPattern(...)`
- ☐ Usa o padrão correto (ex.: "dd/MM/yyyy HH:mm:ss")
- ☐ Aplica o formato no `format(...)` para exibir ou gravar o registro

Pergunta oral: Por que é importante formatar a data? Como ela apareceria sem o formatador?

Nível: ☐ 0 ☐ 1 ☐ 2 ☐ 3 ☐ 4

Pontos: _____

Capacidade 4 · LocalDateTime (Ter)

- ☐ Obtém a data e hora atuais com `LocalDateTime.now()`
- ☐ Registra quando um ambiente foi cadastrado ou alterado
- ☐ Combina o resultado com o `DateTimeFormatter`

Pergunta oral: O que o `now()` devolve e de onde o Java tira essa informação?

Nível: ☐ 0 ☐ 1 ☐ 2 ☐ 3 ☐ 4

Pontos: _____

Capacidade 5 · FileWriter (Ter)

- ☐ Cria o arquivo .txt e grava nele com `FileWriter`
- ☐ Entende a diferença entre **sobrescrever** e **acrescentar** (`append = true`)
- ☐ Fecha o arquivo corretamente (`close()` ou `try-with-resources`)
- ☐ Lê o arquivo de volta para que os dados não se percam ao fechar o programa

Pergunta oral: O que acontece com os dados antigos se abrir o `FileWriter` sem o modo de acrescentar?

Nível: ☐ 0 ☐ 1 ☐ 2 ☐ 3 ☐ 4

Pontos: _____

Capacidade 6 · HashMap (Qua)

- ☐ Cria o `HashMap<String, String>`
- ☐ Usa `put` (cadastrar e alterar), `get` (pesquisar) e `remove` (excluir)
- ☐ Percorre o mapa para listar todos os ambientes (`for` com `entrySet()`)
- ☐ Explica que a **chave não se repete**

Pergunta oral: *O que acontece se eu cadastrar duas vezes a chave F07?*

Analogia: É como o armário da escola: o número do armário (chave) abre uma gaveta (valor); não existem dois armários com o mesmo número.

Nível: ☐ 0 ☐ 1 ☐ 2 ☐ 3 ☐ 4

Pontos: _____

Capacidade 7 · Map (interface) (Qua)

- ☐ Declara usando a interface: `Map<String, String> ambientes = new HashMap<>();`
- ☐ Explica a diferença entre **interface** (`Map`) e **classe** (`HashMap`)
- ☐ Cita ao menos uma outra implementação (ex.: `TreeMap`, `LinkedHashMap`)

Pergunta oral: *Por que declaramos como `Map` e não direto como `HashMap`?*

Analogia: `Map` é a ideia de dicionário (o contrato); `HashMap` é um dicionário específico que cumpre esse contrato.

Nível: ☐ 0 ☐ 1 ☐ 2 ☐ 3 ☐ 4

Pontos: _____

Capacidade 8 · Organização do código (Qui)

- ☐ Indenta o código corretamente (blocos `{ }` alinhados)
- ☐ Usa nomes claros para variáveis e métodos (ex.: `cadastrar`, `listar`)
- ☐ Separa cada opção do menu em um método
- ☐ Comenta as partes principais e mantém o `do while` legível

Pergunta oral: *Se outra pessoa abrir seu código amanhã, ela entende o que cada trecho faz?*

Nível: ☐ 0 ☐ 1 ☐ 2 ☐ 3 ☐ 4

Pontos: _____

5. Checklist de funcionamento do menu

Este quadro **não vale pontos extras**: ele mostra ao aluno onde o programa falhou.

Opção do menu	Funciona?	Observação
Cadastrar	☐ Sim ☐ Parcial ☐ Não	
Listar	☐ Sim ☐ Parcial ☐ Não	
Pesquisar	☐ Sim ☐ Parcial ☐ Não	
Excluir	☐ Sim ☐ Parcial ☐ Não	
Alterar	☐ Sim ☐ Parcial ☐ Não	
Sair (encerra o loop)	☐ Sim ☐ Parcial ☐ Não	
Dados persistem no .txt após fechar	☐ Sim ☐ Parcial ☐ Não	

6. Resultado

Capacidade	Pontos
1. try / catch / finally	
2. Swing	
3. DateTimeFormatter	
4. LocalDateTime	
5. FileWriter	
6. HashMap	
7. Map	
8. Organização do código	
TOTAL	_____ / 24,80

7. Devolutiva

Pontos fortes:

O que treinar para a próxima:

Autoavaliação do aluno (1 a 5)	1 2 3 4 5
Consegui explicar com minhas palavras?	☐ ☐ ☐ ☐ ☐
Consegui testar meu programa sozinho(a)?	☐ ☐ ☐ ☐ ☐

Assinatura do aluno

Assinatura do professor